

Concurrency & async: doing many things without blocking

How to read this file: PART 1 = general, transferable concurrency knowledge (applies to any language/stack). PART 2 = how WCX uses it (gevent greenlets, non-blocking polling). PART 3 = takeaways. **Why this file exists:** concurrency showed up as asides across your notes (gevent, gevent.spawn, non-blocking poll, sleep, generators/ yield, async fetch, "don't block the first token"). It deserves one consolidated mental model — it's the backbone under streaming AND the background writes. **Related:** [[kb-streaming-pipeline]] (generators, non-blocking IO), [[kb-distributed-systems-patterns]], [[kb-storage-model]] (background Cosmos write off the critical path).

PART 0 — WHY THIS MATTERS (philosophy & real-world connection)

Deep principle in one line: most server work is *waiting*, not computing — concurrency is the art of using one thread's idle wait-time to make progress on other work instead of stalling.

Why it exists / what problem it solves. A network/disk/DB call takes milliseconds-to-seconds, almost all of it spent *waiting* for a response, not burning CPU. If your code blocks on every such call, one request monopolizes a whole thread while the CPU sits idle — so serving N concurrent users needs N threads, and threads are expensive (~MB of stack each, kernel scheduling overhead). Concurrency breaks that 1:1 coupling: a single thread juggles thousands of in-flight requests by switching to other work whenever one is waiting. This is *the* reason a modern web server can hold 10,000 open connections on a handful of threads. Without it, every product hits a scaling wall far too early and pays for far too many machines.

Real-world connection beyond this repo. You meet this everywhere: Node.js is built entirely on a single-threaded event loop (its whole identity); Python's `asyncio`, Go's goroutines, Rust's `tokio`, Java's virtual threads (Project Loom), C#'s `async/await` are all the same fight against blocking. Any high-connection server — chat backends, API gateways, proxies, real-time feeds — lives or dies on this. It's also a staple of system-design and language interviews ("how does async/await work?", "processes vs threads vs coroutines?", "what's the GIL?"). And it's the invisible engine under [[kb-streaming-pipeline]]: streaming *requires* non-blocking IO, or one slow stream would freeze every other user.

What breaks without understanding it:

- **Blocking the event loop** — one synchronous CPU-heavy call (or a naive `time.sleep()`) on a single-threaded runtime freezes *every* concurrent request, not just one. The classic Node/async footgun.
- **Reaching for threads when you need processes** (or vice versa) — using threads for CPU-bound work in Python buys nothing (the GIL serializes them); using processes for lightweight IO fan-out wastes memory.
- **Races from assuming atomicity** — even cooperative code can be interrupted at an `await`, so "read-modify-write across an await" can corrupt shared state.
- **Making the user wait on non-essential work** — doing a slow analytics/DB write *inline* on the response path adds latency the user shouldn't pay for (fix: fire it off the critical path).

The transferable mental model. Two orthogonal questions to ask of any workload: (1) **IO-bound or CPU-bound?** — IO-bound wants concurrency (interleave the waits); CPU-bound wants parallelism (more cores/processes). (2) **Cooperative or preemptive?** — cooperative (async/await, greenlets) means "tasks yield at await points → few races but never block the loop"; preemptive (OS threads) means "the scheduler can interrupt anywhere → true parallelism but you must lock shared state." Almost every concurrency decision falls out of those two axes.

PART 1 — GENERAL KNOWLEDGE

1.1 The core problem: blocking

A **blocking** operation stops the current line of execution until it finishes. IO (network, disk, DB) is slow (ms–seconds) but mostly *waiting*, not computing. If a single thread blocks on every network call, it can serve only one request at a time — the CPU sits idle during the wait. Concurrency is about **using that idle wait time to make progress on other work**.

Two distinct goals, often confused:

- **Concurrency** = *dealing with* many things at once (structure: interleave tasks so waiting on one lets another run). Helps IO-bound work.

- **Parallelism** = *doing* many things at once (multiple CPU cores literally executing simultaneously). Helps CPU-bound work. You can have concurrency without parallelism (one core, interleaved) and parallelism without much concurrency structure.

1.2 The models (know the whole ladder)

Model	Unit	Scheduled by	Cost	Good for
Processes	OS process	OS kernel	heavy (own memory)	isolation, true parallelism, CPU-bound (Python multiprocessing sidesteps the GIL)
OS threads	thread	OS kernel (preemptive)	medium (~MB stack each)	parallelism, but shared memory → locks/races
Green threads / coroutines	userland "thread"	a userland scheduler (cooperative)	cheap (KB, thousands)	massive IO concurrency (many connections)
Callbacks / event loop	callback	single-threaded event loop	cheap	IO concurrency (Node.js, JS)
async/await	coroutine	event loop	cheap	modern ergonomic IO concurrency

Preemptive vs cooperative is the key distinction:

- *Preemptive* (OS threads): the scheduler can pause any thread at any instant → true parallelism but you need locks to protect shared state (race conditions).
- *Cooperative* (green threads, async/await): a task runs until it voluntarily **yields** (usually at an `await` /IO point). Between yields it has the runtime to itself → far fewer races, but one task that never yields (a tight CPU loop) **starves everything** ("blocking the event loop").

1.3 The event loop (Node/async model)

A single thread runs a loop: pick a ready task, run it until it hits an `await` on IO, register a callback for when that IO completes, move on to the next ready task. When IO finishes, its continuation is queued. This is how one thread serves thousands of concurrent connections — it's never *waiting*, it's always running whatever is ready. **The rule:** never do slow synchronous/CPU work on the event loop; offload it (worker thread/process) or you block everyone.

1.4 async/await & coroutines

- A **coroutine** is a function that can suspend and resume (like a generator, but for concurrency). `async def` marks one; `await x` means "suspend here until x is ready, and let the loop run other tasks meanwhile."
- `await` ≠ blocking: blocking freezes the thread; `await` frees the thread to do other work. That's the whole point.
- Generators (`yield`) and coroutines are cousins: both suspend/resume by freezing local state. Generators produce values lazily (pull-based, see [[kb-streaming-pipeline]] §1.6); coroutines suspend on IO. Python literally built `async/await` on top of generator machinery historically.
- "Colored functions": async is contagious — calling async code usually requires being async yourself (an `await` chain up to the event loop). A known ergonomic wart.

1.5 Green threads / greenlets (the gevent model)

Some stacks get concurrency **without** rewriting everything as `async / await` by using **green threads**: lightweight userland threads scheduled cooperatively, where blocking IO calls are *monkeypatched* to transparently yield to the scheduler instead of blocking the OS thread.

- **gevent** (Python) = "green + event": greenlets (cheap cooperative userland threads) + an event loop (libev/libuv). You write ordinary-looking blocking code; gevent patches the socket/sleep libraries so that when your code "blocks," it actually yields the greenlet and runs another. Thousands of greenlets fit where thousands of OS threads wouldn't.
- `gevent.spawn(fn, *args)` = start `fn` as a greenlet, running concurrently, returning immediately (don't wait for it). Used to run side work off the critical path.
- Caveat: the greenlets only switch at IO/yield points, and the "hub" (scheduler) must actually be driven — some dev servers don't drive it, so you fall back to running things inline.

- Web servers use this for many-connection workloads: **gunicorn with a gevent worker** runs your app so each request is a greenlet; a request blocked on IO doesn't stop the others.

1.6 Non-blocking IO & "off the critical path"

A recurring pattern: some work (logging, analytics, a secondary DB write) must happen but the user shouldn't *wait* for it. **Fire it concurrently and respond immediately** — the user's latency drops to just the essential path. Requires: (a) the side work is safe to fail (fail-soft: log and tolerate), and (b) any ID/result the response needs is generated locally *before* dispatching the async work. This is "defer X off the streaming/critical path."

1.7 Shared-state hazards (when you DO have parallelism)

- **Race condition:** two tasks read-modify-write shared state interleaved → corruption.
- **Locks/mutexes:** serialize access to shared state; risk deadlock if acquired in inconsistent orders.
- **Double-checked locking:** the "check → lock → check again → build" pattern for lazily initializing a shared singleton exactly once under concurrency (see the client-cache in [\[\[kb-rag-retrieval\]\]](#)).
- Cooperative models (async, greenlets) have *fewer* races because switches only happen at yield points — but they're not race-free (state can change across an `await`).
- **Python GIL:** CPython's Global Interpreter Lock means threads don't give CPU parallelism for pure-Python code (only one runs Python bytecode at a time); threads still help IO-bound work (they release the GIL during IO). For CPU parallelism use processes.

PART 2 — WORKED EXAMPLE: concurrency in WCX

- **gunicorn gevent worker** runs the portal backend: each incoming chat request is a greenlet. This is *why* the streaming poll loop's `sleep(0.8)` between polls is non-blocking — under gevent, `sleep` yields the greenlet so other requests progress; it doesn't freeze the worker. (§1.5)
- **Background session write off the critical path** (§1.6): `gevent.spawn(store.persist_session_doc, doc)` kicks off the Cosmos write (50–200ms network) concurrently so the user gets their first token immediately instead of waiting on the write. The session `id` is generated locally *first* so the response doesn't need the write to finish. Fail-soft: a failed write is logged and tolerated, chat proceeds. Under the debug dev server the gevent hub isn't driven, so it runs inline instead: `if get_debug_flag(): persist inline else: gevent.spawn(...)`.
- **Generators for streaming** (§1.4, and [\[\[kb-streaming-pipeline\]\]](#) §1.6): `stream()` is a generator; each `yield` hands out one SSE frame and pauses, so the answer streams instead of materializing whole.
- **Frontend async/await** ([\[\[kb-frontend-architecture\]\]](#)): `await reader.read()` suspends the JS event loop task until the next byte-block arrives, freeing the browser to stay responsive — the browser-side of §1.3/§1.4.

PART 3 — transferable takeaways

1. Concurrency (interleave, IO-bound) ≠ parallelism (multi-core, CPU-bound). Pick the model by which you need.
2. Cooperative models (async/await, greenlets) trade "you must never block the loop" for "far fewer races." Preemptive threads trade "true parallelism" for "you must lock shared state."
3. `await` / `yield` free the thread; blocking freezes it. Never do slow sync/CPU work on an event loop.
4. Greenlets (gevent) give many-connection concurrency without rewriting to async — by patching blocking calls to yield.
5. Push non-essential work **off the critical path** (fire-and-forget concurrently, fail-soft, generate needed IDs first) to cut user-visible latency.
6. Generators and coroutines are the same suspend/resume idea — one for lazy values, one for IO.